

An Introduction to Software Engineering Ethics

MODULE AUTHORS:

Shannon Vallor, Ph.D. Associate Professor of Philosophy, Santa Clara University

SPECIAL CONTRIBUTOR TO INTRODUCTION:

Arvind Narayanan, Ph.D. Assistant Professor of Computer Science, Princeton University

These documents contain fields that can be filled in by users who have downloaded the free Adobe Reader. Simply download the appropriate document to your computer, type your comments in the boxes, and save the completed version. To send the version with your responses, include it as an attachment to an email.

What do we mean when we talk about ‘ethics’?

Ethics in the broadest sense refers to the concern that humans have always had for figuring out *how best to live*. The philosopher Socrates is quoted as saying in 399 B.C., “the most important thing is not life, but the good life.”¹ We would all like to avoid a life that is shameful and sad, wholly lacking in achievement, love, kindness, beauty, pleasure or grace. Yet what is the best way to achieve the opposite of this – a life that is not only acceptable, but even excellent and worthy of admiration? This is the question that the study of ethics attempts to answer.

Today, the study of ethics can be found in many different places. As an academic field of study, it belongs primarily to the discipline of philosophy, where scholars teach and publish research about the nature and structure of ethical norms. In community life, ethics is pursued through diverse cultural, political and religious ideals and practices. On a personal level, it can be expressed in an individual’s self-reflection and continual strivings to become a better person. In work life, it is often formulated in formal codes or standards to which all members of a profession are held, such as those of medical ethics. Professional ethics is also taught in dedicated courses, such as business ethics. It can also be infused into courses such as this one.

What is ethics doing in a course for software engineers?

Like medical, legal and business ethics, **engineering ethics** is a well-developed area of professional ethics in the modern West. The first codes of engineering ethics were formally adopted by American engineering societies in 1912-1914. In 1946 the National Society of Professional Engineers (NSPE) adopted their first formal [Canons of Ethics](#). In 2000 ABET, the organization that accredits university programs and degrees in engineering, began to [formally require](#) the study of engineering ethics in all accredited programs: “Engineering programs must demonstrate that their graduates have an understanding of professional and ethical responsibility.”² Professional engineers today, then, are expected to both *learn about* and *live up to* ethical standards as a condition of their membership in the profession.

¹ Plato, *Crito* 48b. In Cahn (2010).

² ABET 2000 criterion 3(f) (ABET, 1998).

But the average computer/software engineering student might still be confused about how and why this requirement should apply to them. Software engineering is a relatively young practice and compared with other engineering disciplines, its culture of professionalism is still developing. This is reinforced by the fact that most engineering ethics textbooks focus primarily on ethical issues faced by civil, mechanical or electrical engineers. The classic case studies of engineering ethics depict catastrophic losses of life or injury as a result of ethical lapses in these fields: the Challenger explosion, the Ford Pinto fires, the Union Carbide/Bhopal disaster, the collapse of the Hyatt walkway in Kansas City. When we think about the engineer's most basic ethical duty to "hold paramount the safety, health, and welfare of the public,"³ it is clear why these cases are chosen - they powerfully illustrate the importance of an engineer's ethical obligations, and the potentially devastating consequences of failing to live up to them.

But software engineers build *lines of code*, not cars, rockets or bridges full of vulnerable human beings. Where is the comparison here? Well, one answer might already have occurred to you. How many cars or rockets are made today that do *not* depend upon critical software for their safe operation? How many bridges are built today without the use of sophisticated computer programs to calculate expected load, geophysical strain, material strength and design resilience? A failure of these critical software systems can result in death or grievous injury just as easily as a missing bolt or a poorly designed gas tank. This by itself is more than enough reason for software engineers to take seriously the ethics of their professional lives. Is it the *only* reason? What might be some others? Consider the following:

The software development and deployment process in the Internet era has some peculiarities that make the ethical issues for software engineers even *more* acute in some ways than for other types of engineers. First, the shortened lifecycle has weakened and in some cases obliterated software review by management and legal teams. In the extreme, for Web applications like Facebook, it is normal for individual engineers or small groups of engineers to code and deploy features directly, and indeed the culture takes pride in this. Even where more traditional development practices prevail, at least some deployments like bug fixes are shipped with only technical (and not ethical) oversight. At any rate, engineers at least retain the *ability* to deploy code directly to end users, an ability that can easily be abused.

All of this is in stark contrast to say, a civil engineering project with a years-long (or decades-long) lifecycle and multiple layers of oversight. Nor does such a project offer a malicious engineer any real means to obfuscate her output to sneak past standards and safety checks.

Second is the issue of scale, perhaps the defining feature of the software revolution. Typically the entire world is part of the addressable market. Of course, it is scale that has led to the potential for individual engineers to create great good, but with it naturally comes the ability to cause great harm, especially when combined with the first factor above.

³ [NSPE Code of Ethics for Engineers, First Fundamental Canon.](#)

Here's a rather benign but illustrative example. On June 9, 2011, Google released a "doodle" honoring Les Paul which users found addictive to play with. This is a type of project that's typically done by an individual engineer on their "20% time" in a day or two. A third party, RescueTime, estimated that [5.3 million hours](#) were spent playing this game.⁴ Let us pause to consider that 5.3 million hours equates to about *eight lifetimes*.

Did the doodle make a positive contribution to the world? Do engineers at Google have an obligation to consider this question before releasing the feature? What principle(s) should they use to determine the answer? These are all valid questions, but what is perhaps even more interesting here is the disproportionality between the amount of time engineers spent creating the feature (at most a few person-days, in all likelihood), and the amount of time users spent on it (several lifetimes). Often, in today's world, engineers must grapple with these questions instead of relying on management or anyone else.

Finally, the lack of geographic constraints means that engineers are generally culturally unfamiliar with some or most of their users. The cost-cutting imperative often leaves little room for user studies or consultations with experts that would allow software development firms to acquire this familiarity. This leads to the potential for privacy violations, cultural offenses, and other such types of harm.

For example, people in many countries are notoriously sensitive to the representation of disputed border territories on maps. In one recent example, an error in Google maps led to Nicaragua dispatching forces to its border with Costa Rica. Google then worked with US State Department officials to correct the [error](#).⁵

On top of these considerations, software engineers share with everyone a basic human desire to flourish and *do well* in life and work. What does that have to do with ethics? Imagine a future where you are faced with a moral quandary arising from a project you are working on that presents serious risks to users. In that scenario, will you act in a way that you would be comfortable with if it later became public knowledge? Would it matter to you whether your family was proud or shamed by your publicly exposed actions? Would it matter to you whether, looking back, you saw this as one of your *better* moments as a human being, or one of your *worst*? Could you trust anyone to whom these outcomes *didn't* matter?

Thus ethical obligations have both a *professional* and a *personal* dimension. Each are essential to consider; without a sense of personal ethics, one would be indifferent to their effect on the lives of others in circumstances where one's professional code is silent. To understand what's dangerous about this, consider any case in human history when a perpetrator of some grossly negligent, immoral or inhumane conduct tries to evade their responsibility by saying, 'I was just following orders!' So personal ethics helps us to be sure that we take full responsibility for our moral choices and their consequences.

⁴ ["Google Doodle Strikes Again! 5.3 Million Hours Strummed."](#) Rescue Time, Jun 9 2011.

⁵ ["Google Maps Embroiled in Central America Border Dispute."](#) AFP, Nov 6 2010.

But for professionals who serve the public or whose work impacts public welfare, a personal code of ethics is just not enough. Without a sense of professional ethics, one might be tempted to justify conduct in one's own mind that could never be justified in front of others. Additionally, professional ethics is where one learns to see how broader ethical standards/values (like honesty, integrity, compassion and fairness) apply to one's particular type of work. For example, wanting to have integrity is great – but what does integrity look like in a software engineer? What sort of specific coding practices demonstrate integrity, or a lack of it? This is something that professional codes of ethics can help us learn to see. Finally, being a professional means being a part of a moral community of others who share the same profound responsibilities we do. We can draw strength, courage, and wisdom from those members of our professional community who have navigated the same types of moral dilemmas, struggled with the same sorts of tough decisions, faced up to the same types of consequences, and ultimately earned the respect and admiration of their peers and the public.

Broadening our view of software engineering ethics

Certainly, software engineers must concern themselves primarily with the health, safety and welfare of those who are affected by their work, as the so-called 'paramountcy clause' of NSPE's Code of Ethics states. But we need to broaden our understanding of a number of aspects of this claim, including:

- The **types** of harms the public can suffer as result of this work;
- How software engineers **contribute** to the good life for others;
- **Who** exactly are the 'public' to whom the engineer is obligated;
- **Why** the software engineer is obligated to protect the public;
- What **other** ethical obligations software engineers are under;
- **How** software engineers can actually live up to ethical standards;
- What is the end **goal** of an ethical life in software engineering;
- What are the professional **codes** of software engineering ethics;

Let's begin with the first point.

PART ONE

What kinds of harm to the public can software engineers cause? What kinds of harm can they prevent?

We noted above that failures of critical software systems can result in catastrophic loss of life or injury to the public. If such failures result, directly or indirectly, from software engineers' choices to ignore their professional obligations, then these harms are clearly the consequences of *unethical* professional behavior. Those responsible each bear the moral weight of this avoidable human suffering, whether or not this also results in legal, criminal or professional punishment.

But what *other* kinds of harms do software engineers have an ethical duty to consider, and to try to prevent? Consider the following scenario:

Case Study 1

Mike is a father of 3, and in order to save for their college educations, he has been working two jobs since his kids were born. His daughter Sarah has worked as hard as she can in high school to get high grades and SAT scores; as a result of her hard work she has been accepted to a prestigious Ivy-League college, and the deposit for her first year is due today. If the deposit goes unpaid, Sarah loses her spot in the freshman class. Mike paid the bill last week, but today he gets an email from the college admissions office saying that his payment was rejected for insufficient funds by his bank, and if he does not make the payment by the end of the day, Sarah will lose her place and be unable to attend in the Fall. Panicked, Mike calls the bank – he had more than enough money in his savings to cover the bill, so he cannot understand what has happened. The bank confirms that his account had plenty of funds the day before, but cannot tell him why the funds are gone now or why the payment was rejected. They tell him there must be some 'software glitch' involved and that they will open an investigation, but that it will take weeks to resolve. They will only restore the funds in his account once the investigation is completed and the cause found. Mike has no other way to get the money for the deposit on such short notice, and has to tell Sarah that he couldn't cover the bill despite his earlier promise, and that she won't be attending college in the Fall.

Question 1.1:

What kinds of harm has Mike probably suffered as a result of this incident? What kinds of harm has Sarah probably suffered? (Make your answers as full as possible; identify as many kinds of harm done as you can think of).

Type your answer here

Question 1.2:

Could the problem with Mike's account have been the result of an action (or a *failure* to perform an action) by a software engineer? How many possible scenarios/explanations for this event can you think of that involve the conduct of one or more software engineers? Briefly explain the scenarios:

Type your answer here (continue answer on next page if needed)

(Continue your answer to 1.2 from previous page)

Question 1.3:

Taking into account what we said about ethics in the introduction, could any of the scenarios you imagined involve an *ethical* failure of the engineer(s) responsible? How? Explain:

***Note:** An ethical failure would be preventable, and one that a good human being with appropriate professional care and concern *would* and *should* have prevented (or at least have made a serious effort to prevent).

Type your answer here

Let's try a different scenario:

Case Study 2

Karen is a young lawyer at a prestigious firm with an incredibly hectic and stressful schedule, who needs to organize what little free time she has more efficiently. She has just downloaded a new app called Errand Whiz onto her iPhone; this app merges information from Karen's to-do list, information on her purchasing habits from retail stores she shops at, and GPS software to produce the most efficient map and directions for running errands on her days off. Based on what it knows about what she needs to purchase and her general shopping habits, it tells Karen what locations of her favorite stores to visit on a given day, in what order and by what routes – this way she can get her errands done in the least amount of time, traveling the least number of miles. To accomplish this, the app aggregates information not only about where she lives and shops, but also tracks what she typically buys in each store, how much she buys, what she typically pays for each item. This collected data is not stored on Karen's phone, but on a separate server that the app links to when it needs to create a shopping map. The app encourages users to log in via Facebook, as the developers have made a deal with Facebook to sell this data to third-party advertisers, for the purpose of targeting Facebook ads to Karen and her friends.

Question 1.4: In what ways could Karen potentially be harmed by this app, depending on how it is designed and how her shopping data is handled and used? Identify a few harmful scenarios you can think of, and the *types* of harm she could suffer in each:

Type your answer here (continue on next page if needed)

(Continue your answer to 1.4 from previous page)

Question 1.5: Which if any of these harms could result from ethical failings on the part of the people who developed Errand Whiz? *How*, specifically?

Type your answer here (continue on next page if needed)

(Continue your answer to 1.5 from previous page)

Question 1.6: What actions could the people behind Errand Whiz take to prevent these harms? Are they ethically *obligated* to prevent them? Why or why not? Explain your answer.

Type your answer here

Ideally, these scenarios have helped to broaden your understanding of the ethical scope of software engineering. In considering and protecting the ‘health, safety and welfare’ of the public, we must not limit our thinking to those contexts in which our design choices or coding practices have the potential to cause someone’s death, or cause them direct physical injury. *The harms that people can suffer as a result of failures by software engineers to consider their ethical obligation to the public are far more numerous and more complex than we might think.*